

DL for ENG

LECTURE 7.1

Physics-Informed Neural Networks

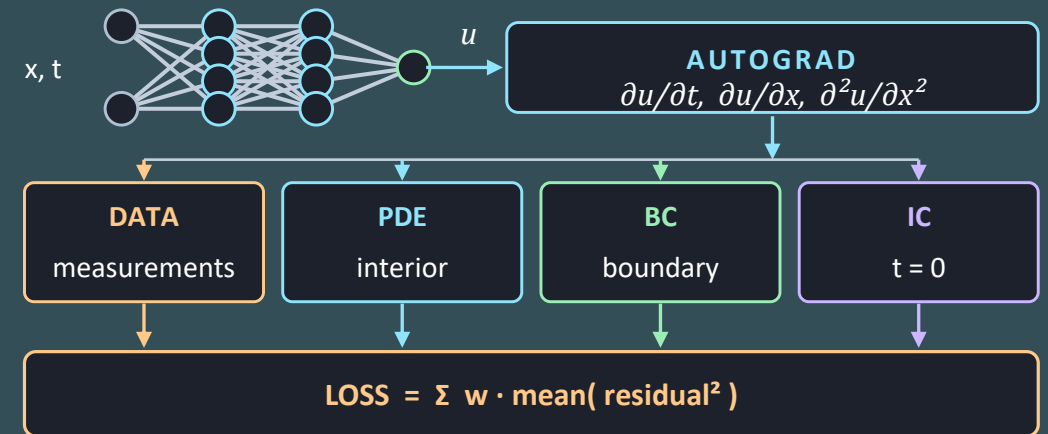
Remus Teodorescu

With support from Research Assistant Noman Khan -
nomank@energy.aau.dk

Aalborg University · ret@et.aau.dk

Deep Learning for Engineering · 2026

ANSATZ · RESIDUAL · CONDITIONS



a gradient step on the weights, then again

A physics-informed network is a dense network that is the solution of a differential equation: autograd gives its derivatives, and the loss is the equation's residual at sampled points plus its boundary and initial conditions.

Learning Objectives

SECTIONS, IN ORDER

- 1 **PINN** — what it is, the solution ansatz, smooth activations
- 2 **Formulation** — the operator form, boundary and initial conditions, forward and inverse
- 3 **Loss** — the composite loss, soft and hard conditions, weights, collocation

Autograd and the optimisers, Adam then L-BFGS, are L6.1's: this lecture uses them without teaching them again.

READ, AND WHAT EACH ANSWERS

- | | |
|--------------------------|--|
| Liu ch. 2.1 | what is a PINN, and what can it represent? |
| Liu ch. 2.2 | which equation, conditions and unknowns? |
| Liu ch. 2.2 – 2.9 | how does each condition enter the loss? |

Recap

1 – 5 · PRE-TRAINING AND TRANSFER

- 1 What is transfer learning, and why do the first layers of a trained network carry over to a new task? *UDL ch. 9.3.6*
- 2 What does a transferred network compute: which layers are kept, which are replaced, and why? *UDL ch. 9.3.6; Ex_06 notebook 03*
- 3 How is it fine-tuned: what do you unfreeze, at what learning rate, and what is catastrophic forgetting? *UDL ch. 9.3.6, 12.6; Ex_06 notebook 03*
- 4 How did transfer compare with training from scratch in notebook 03, and when would it stop helping? *Ex_06 notebook 03*
- 5 How is the model you start from made, and what does continued pre-training on your own unlabelled data add? *UDL ch. 9.3.7*

6 – 10 · REINFORCEMENT TRAINING

- 6 What kind of problem is reinforcement learning, and how do state, action, policy and reward map onto control? *UDL ch. 19.1*
- 7 What does the policy compute, what is the return it maximises, and why discount it by γ ? *UDL ch. 19.1 – 19.2*
- 8 How is a policy trained: how does the policy gradient get a gradient from a world you cannot differentiate? *UDL ch. 19.5*
- 9 How does reinforcement learning differ from supervised learning, what is credit assignment, and where does RL not fit? *UDL ch. 19*
- 10 What is it used for in energy, and when is imitating an expert, such as an MPC, the better choice? *UDL ch. 19.7*

Physics-Informed Network

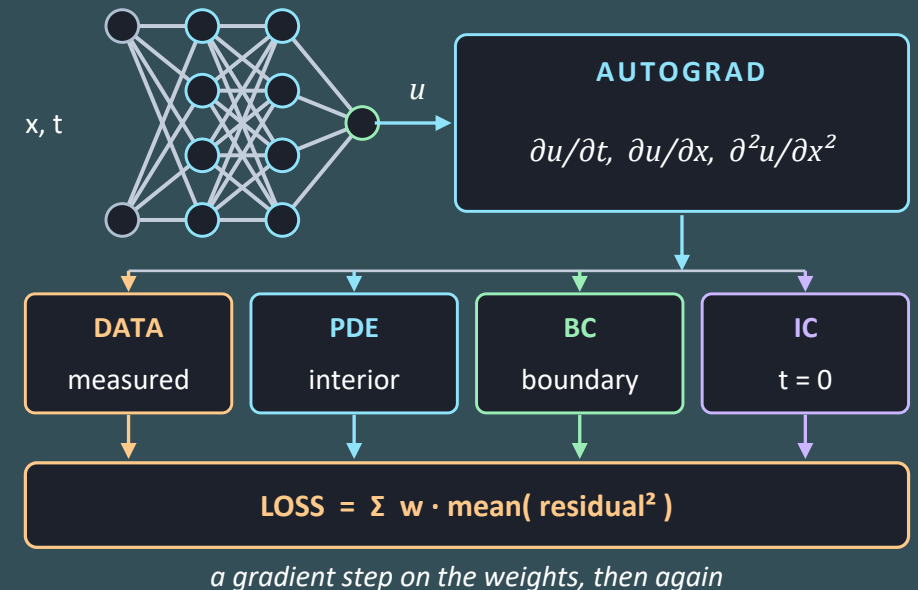
Physics-informed network — a neural network trained to be the solution of a differential equation: it takes the coordinates x and t and outputs the field u there. Its loss is the equation itself: autograd gives the derivatives of u , and the residual of the equation, squared and averaged over sampled points, is minimised together with the boundary and initial conditions.

AGAINST A MESH-BASED SOLVER

	FEM, FD	PINN
the unknowns	nodal values	the network's weights
the derivatives	a stencil or an element	autograd, exact
the geometry	meshed	sampled

TALK · 2 MIN How would you explain what a physics-informed network is, in your own words? Then, in the figure on the right, where do the equation and its conditions enter, and what does the data term add?

ONE NETWORK, ONE LOSS



No labels are needed: the equation and its conditions are the targets.

Solution Ansatz

Solution ansatz — the network written as the approximate solution, $u(\mathbf{x}, t) \approx N(\mathbf{x}, t; \theta)$. A wide enough network can come as close as needed to any continuous solution, and the interior sample needs at least N^* points, a number set by the neurons rather than by the weights.

THE NETWORK IS THE SOLUTION

$$u(\mathbf{x}, t) = \mathcal{N}_\theta(\mathbf{x}, t)$$

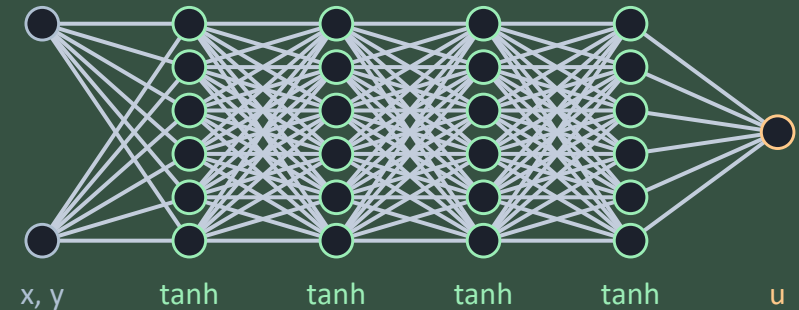
N the network, ϑ its parameters, u the field it stands for

- **one network** — u at any point, no interpolation
- **N^*** — counts neurons, not weights

THE ANSATZ IN PYTORCH

```
net = nn.Sequential(nn.Linear(2, 32), nn.Tanh(),          # (x, y) in, 32 wide
                    *[m for _ in range(3) for m in (nn.Linear(32, 32), nn.Tanh())], # 3 more
                    nn.Linear(32, 1))                  # u out
u = net(xy)                                           # u(x, y), at any point
```

EX_07.1: FOUR LAYERS OF 32



N^* AGAINST THE WEIGHTS

network	inputs	N^*	weights
32×4	2	135	3 297
100×2	3	206	10 601
200×4	3	808	121 601

Ex_07.1 samples 2 000 points: $15 \times N^$. Liu writes it P^* .*

Smooth Activation Functions

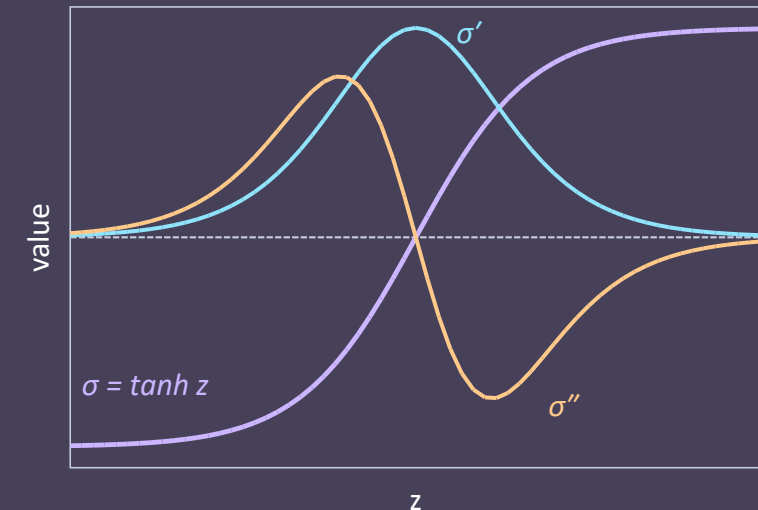
Smooth activation functions — a residual with k -th derivatives needs an activation whose k -th derivative exists and is not zero. $\tanh$ and $\sin$ are smooth to every order; ReLU has a zero second derivative almost everywhere, so a second-order residual built on it vanishes whatever the network.

THE SECOND DERIVATIVE, BY ACTIVATION

activation	$\sigma''(z)$	second-order PDE
$\tanh$	$-2 \tanh z (1 - \tanh^2 z)$	the default
$\sin$	$-\omega^2 \sin(\omega z)$	oscillations
sigmoid	$\sigma (1 - \sigma)(1 - 2\sigma)$	saturates; avoid
ReLU	0 almost everywhere	no

- **the order of the equation** — sets the smoothness
- **$\tanh$** — the default in every exercise of Part 2

TANH AND ITS FIRST TWO DERIVATIVES



All three are smooth. Amber is $\tanh$'s second derivative; ReLU's would be flat zero — a Laplacian of nothing.

Operator Form

Operator form — one equation for every problem: $\mathcal{F}(u; \mathbf{x}, t, \mathbf{c}) = 0$ in the domain Ω , where $\mathcal{F}$ is a differential operator acting on the field u and $\mathbf{c}$ are the physical parameters. The residual is $\mathcal{F}$ with the network in place of u .

ONE EQUATION FOR EVERY PROBLEM IN PART 2

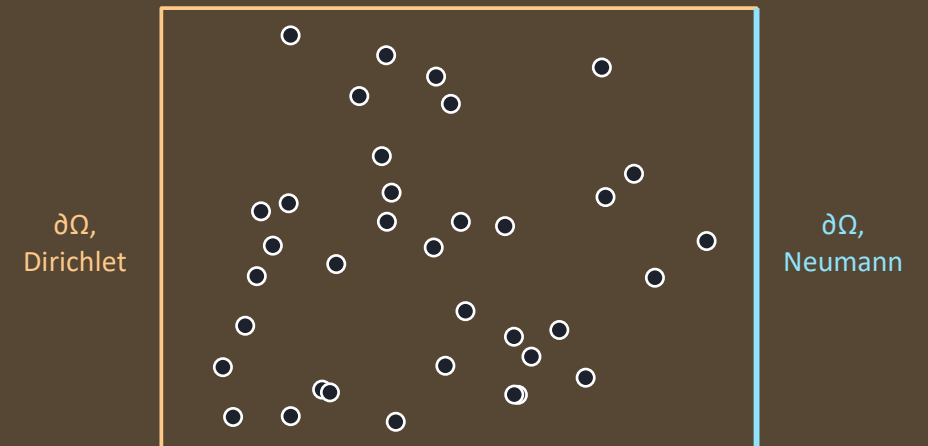
$$\mathcal{F}(u; \mathbf{x}, t, \mathbf{c}) = 0, \quad \mathbf{x} \in \Omega, \quad t \in [0, t_{\text{end}}]$$

Poisson $\mathcal{F} = u_{xx} + u_{yy} + f$

THE RESIDUAL IN PYTORCH

```
xy.requires_grad_(True); u = net(xy)           # the field at the points
du = torch.autograd.grad(u.sum(), xy, create_graph=True)[0]  # ∇u
u_xx = torch.autograd.grad(du[:, 0].sum(), xy, create_graph=True)[0][:, 0]
F = u_xx + u_yy + f                             # Poisson's residual; u_yy likewise
```

THE DOMAIN AND ITS BOUNDARY



F is checked at the white points, the conditions on the edges: a value on the amber walls, a derivative on the cyan one.

Boundary and Initial Conditions

Boundary and initial conditions — the extra equations that make a problem well posed: a Dirichlet condition fixes the value on part of the boundary, a Neumann condition its normal derivative, an initial condition the field at $t = 0$, and an initial velocity its rate at $t = 0$ when the equation is second order in time.

FOUR CONDITIONS, FOUR RESIDUALS

DBC $u(\mathbf{x}, t) = \bar{u}(\mathbf{x}, t) \quad \forall \mathbf{x} \in \partial\Omega_{DBC}$

NBC $\mathcal{B}(\mathbf{x}, t, \mathbf{c}) = 0 \quad \forall \mathbf{x} \in \partial\Omega_{NBC}$

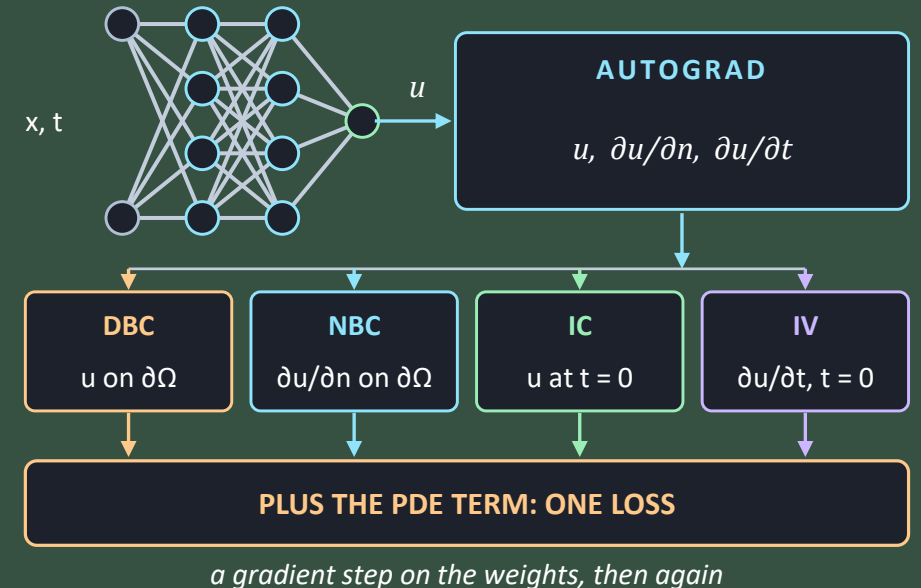
IC $\mathcal{I}(\mathbf{x}, 0, \mathbf{c}) = 0 \quad \forall \mathbf{x} \in \Omega$

IV $\mathcal{V}(\mathbf{x}, 0, \mathbf{c}) = 0 \quad \forall \mathbf{x} \in \Omega$

THE CONDITIONS IN PYTORCH

```
B_d = net(x_wall) - u_bar           # Dirichlet: the value
B_n = (grad(u, x_flux) * n).sum(1) - g # Neumann: ∂u/∂n; grad = autograd
I = net(x_t0) - u_0                 # initial: u at t = 0
V = grad(u, x_t0)[: , -1:] - v_0    # velocity: ∂u/∂t at t = 0
```

WHERE EACH CONDITION ENTERS



Poisson: walls only · heat: walls and IC · wave: walls, IC and IV.

Forward and Inverse Problems

Forward and inverse problems — in a forward problem the equation and its parameters are known and the network finds the field. In an inverse problem some parameters c are unknown: they become trainable beside the weights, and sparse measurements, added to the loss as a data term, identify them.

TWO SETTINGS, ONE LOOP

$$u_t + \mathcal{N}[u; c] = 0$$

a time derivative plus an operator with parameters c ; Raissi writes λ

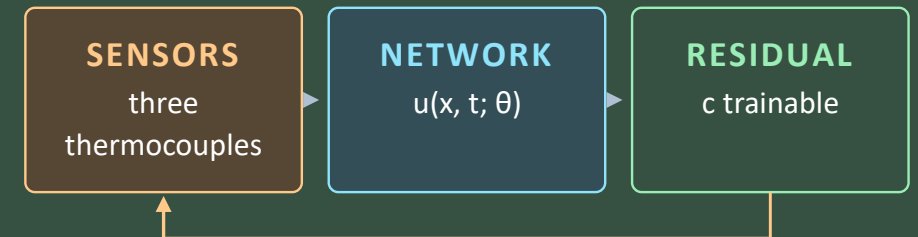
- **forward** — c known, find the field u
- **inverse** — c trainable; the data identifies it

AN INVERSE PROBLEM IN PYTORCH

```
c = nn.Parameter(torch.tensor(1.0))          # c, the unknown parameter
opt = torch.optim.Adam([*net.parameters(), c]) # trained beside  $\theta$ 
loss = Fres(net, x, c).pow(2).mean() + (net(x_s) - u_s).pow(2).mean() # + data
```

THE TWO SETTINGS

	forward	inverse
given	F, c , conditions	F , sparse u
unknown	u	u and c
data term	absent	essential
cost	one run per c	one run



the gradient reaches c

One run recovers the field and the unknown coefficients together.

Composite Loss

Composite loss — the sum of one mean squared residual per equation: the PDE on interior points, each condition on its own boundary or initial points, and the measurements if there are any, each multiplied by a weight. Every term vanishes at the exact solution.

SIX TERMS, EACH A MEAN SQUARED RESIDUAL

$$\mathcal{L} = \mathcal{L}_{data} + w_{PDE}\mathcal{L}_{PDE} + w_{DBC}\mathcal{L}_{DBC} + w_{NBC}\mathcal{L}_{NBC} + w_{IC}\mathcal{L}_{IC} + w_{IV}\mathcal{L}_{IV}$$

$$\mathcal{L}_{PDE} = \frac{1}{n_{PDE}} \sum_{i=1}^{n_{PDE}} [\mathcal{F}_{\theta}(\mathbf{x}_i, t_i)]^2$$

the PDE term: the residual, squared, averaged over n interior points

THE COMPOSITE LOSS IN PYTORCH

```
L_pde = F.res.pow(2).mean()           # interior points
L_dbc = B_d.pow(2).mean(); L_ic = I.pow(2).mean() # their own points
loss = w_pde * L_pde + w_dbc * L_dbc + w_ic * L_ic # one number
```

- **every term** — zero at the exact solution

WHERE EACH TERM IS EVALUATED

L_data measurements, if any

L_PDE interior collocation points

L_DBC values on the boundary

L_NBC derivatives on the boundary

L_IC the field on $t = 0$

L_IV its rate on $t = 0$

Soft Enforcement

Soft enforcement — a condition imposed as a penalty term in the loss: the network is asked to meet it at sampled boundary or initial points, competing with the PDE term for the same weights, so it holds only approximately. Neumann and initial conditions are added the same way.

THE CONDITION AS A PENALTY

$$\mathcal{L}_{DBC} = \frac{1}{n_{DBC}} \sum_{i=1}^{n_{DBC}} [u_{\theta}(\mathbf{x}_i, t_i) - \bar{u}(\mathbf{x}_i, t_i)]^2$$

the prescribed value $\bar{u}$ against the network's output on n boundary points

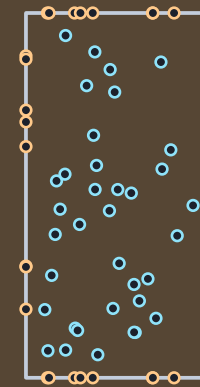
- **it competes** — with the PDE term, for the same weights
- **met approximately** — its loss weight sets how closely

A SOFT WALL IN PYTORCH

```
L_dbc = (net(x_wall) - u_bar).pow(2).mean() # the wall, sampled
loss = L_pde + w_dbc * L_dbc               # a request, at rate w
```

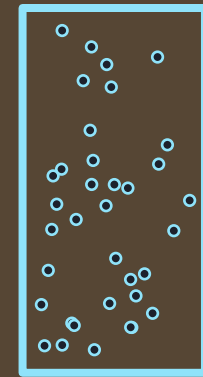
THE SAME WALL, ASKED OR BUILT IN

SOFT



$loss + w \cdot (u - \bar{u})^2$
at the wall: $u \approx \bar{u}$

HARD



$u = A + D \cdot N$
at the wall: $D = 0, u = \bar{u}$

Soft asks for the wall at the amber points and meets it nearly. Hard multiplies the network by D , zero on every wall, and meets it exactly.

Hard Enforcement

Hard enforcement — a condition built into the network's output so that it holds exactly: $u = A(x) + D(x) \cdot N(x; \theta)$. A has the right value on the boundary and D is zero there, so the network cannot change it. The penalty term disappears; the price is a new D for every geometry.

THE CONDITION BUILT INTO THE NETWORK

$$u_{\text{trial}}(x, y) = x(1 - x) y(1 - y) \mathcal{N}_{\theta}(x, y)$$

the trial solution: network $\times$ mask D , so $u = 0$ on all four edges

- **D** — a mask: zero on the wall, free inside
- **the boundary term** — gone, nothing left to balance
- **the price** — a new D for every geometry

TALK · 2 MIN How would you explain the difference between a soft and a hard condition, in your own words? Then, in the table on the right, why is the hard network more accurate inside the square as well as on its walls?

MEASURED · POISSON, UNIT SQUARE

	soft, w = 1	soft, w = 100	hard
relative L_2	9.9e-3	7.0e-4	3.2e-5
max $ u $, wall	2.2e-2	1.5e-3	0
PDE residual	3.8e-4	7.2e-4	6.5e-5

L_2 : the error over the square as a fraction of the true field, $1e-2 = 1\%$. Wall: the largest $|u|$ where u should be 0. Residual: how far the equation is from holding. $\tanh 40 \times 2$, 320 interior, 240 boundary, seed 0.

A HARD WALL IN PYTORCH

```
def u(xy):
    x, y = xy[:, :1], xy[:, 1:]
    return x*(1-x)*y*(1-y) * net(xy)
loss = Fres(u, xy).pow(2).mean()
```

trial solution: D times N
the coordinates
zero on all walls
the PDE term only

Loss Weights

Loss weights — the multipliers that set how much each loss term counts. The terms differ in units and size, and the optimiser attends to the largest, so a weight is a rate of exchange between them: set by hand, rebalanced from gradient norms, or removed by hard enforcement.

TERMS IN DIFFERENT UNITS, ONE OPTIMISER

$$w_k \leftarrow \frac{|\nabla_{\theta} \mathcal{L}_{PDE}|}{|\nabla_{\theta} \mathcal{L}_k|}$$

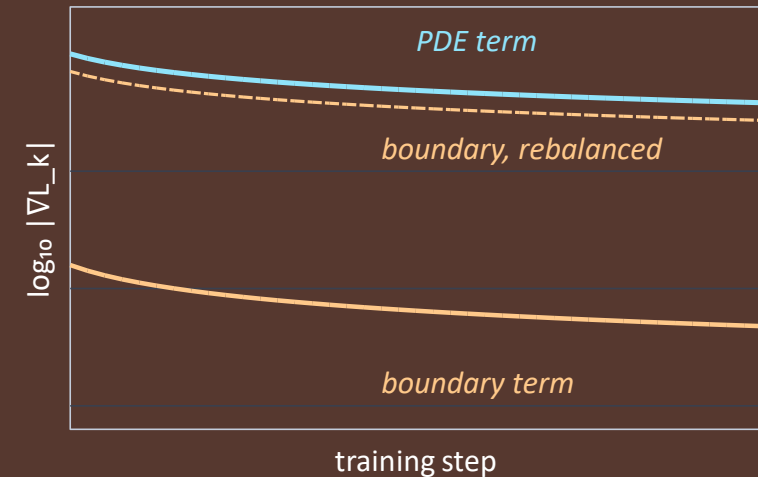
gradient-norm balancing: lift each term until its gradient matches the PDE term's

- **the largest term** — the one the optimiser follows
- **hard enforcement** — removes the term to weigh

BALANCING IN PYTORCH

```
g_pde = torch.autograd.grad(L_pde, w, retain_graph=True) # w: the weights
g_dbc = torch.autograd.grad(L_dbc, w, retain_graph=True) # the wall's gradient
w_dbc = norm(g_pde) / norm(g_dbc) # lift the wall to match
```

GRADIENT NORM PER TERM, SCHEMATIC



Schematic: the PDE gradients are one to two orders larger, so the optimiser ignores the conditions. Balancing lifts amber to the dashed line.

Collocation

Collocation — evaluating the residual at a finite set of sampled points, the collocation points, in the domain and on its boundary. Between the points nothing is imposed, so their number and placement set the accuracy: uniform, random, or refined where the residual is largest.

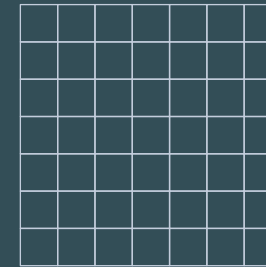
THE RESIDUAL AT SAMPLED POINTS

- **between the points** — nothing is imposed
- **at least N^* interior points** — more is better
- **Ex_07.1** — 2 000 inside, 320 on the walls

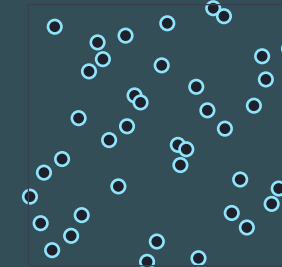
COLLOCATION IN PYTORCH

```
x_in = torch.rand(320, 2)           # interior points, at random
F_c = residual(net, cand).abs()      # the residual at candidates
x_in = torch.cat([x_in, cand[F_c.topk(30).indices]]) # refine where it is large
```

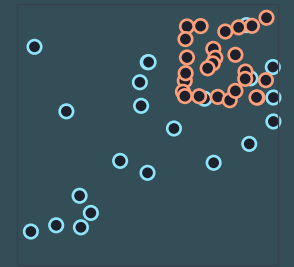
THREE WAYS TO PLACE THE POINTS



uniform grid



Latin hypercube



residual-adaptive

grid

regular; the network sees its pattern

Latin hypercube

one point per row and column

adaptive

points added where $|F|$ is largest

Orange is what refinement adds.

Advanced Topics

RELATED TOPICS LEFT OUT, AND WHERE THEY MATTER

topic	what it adds	read	where it makes a difference
energy method	an energy, first derivatives only	Liu ch. 2.8	elasticity, structures
the stencil	finite differences on the network	Liu ch. 2.7.6	costly derivatives
learned boundary	a second network for curved walls	Liu ch. 3.1 – 3.3	holes, curved geometry
adaptive sampling	points where the residual is large	Liu ch. 5.6	shocks, thin layers
periodic activations	sine networks for oscillations	Liu ch. 5.7.4	waves, vibration

WHY THEY ARE LEFT OUT

Each answers a difficulty the problems of this block do not meet. A plain PINN, with soft or hard conditions, solves Ex_07.1; the rest return when a problem asks for them.

Exercise 7.1

THE NOTEBOOKS, IN ORDER

- **00** — checks the setup: autograd against a hand derivative
- **01** — the slot's walls as a penalty, and a sweep of the weight
- **02** — the walls built into the network
- **03** — both on a sparse sample, and the report

THE QUESTION EACH NOTEBOOK ANSWERS

- 01** how far does a soft wall miss?
- 02** what does a hard wall buy?
- 03** which holds up with few points?

REPORT QUESTION, TO CONCLUDE

The soft-wall error plateaus; the hard-wall error does not. Which would you use with no exact solution, and what would you check instead?

Key Takeaways

PINN

what it is	a network that is the solution field $u(x, t)$
function	coordinates in, u out; derivatives by autograd
train, tune	tanh, at least N^* points, Adam then L-BFGS
vs FEM	weights, not nodal values; no mesh
used for	forward and inverse problems, all of Part 2

PINN LOSS

what it is	one mean squared residual per equation
function	soft: each condition asked at its own points
train, tune	weights as rates of exchange; balance them
vs soft	hard: exact on the wall, and more accurate
used for	every block of Part 2, a different residual

Next — L7.2: the fundamental equations, and the conditions each one needs.

Questions

1 – 5 · PINN

- 1 What is a physics-informed network: what does it take in and give out, and how does its loss differ from a supervised one? *Liu ch. 2.1*
- 2 What function does it compute: what does the ansatz promise, and how many collocation points does N^* ask for? *Liu ch. 2.1, 2.5.2; Ex_07.1 notebook 01*
- 3 How is it trained and tuned: why must the activation be smooth to the order of the equation, and what does ReLU do? *Liu ch. 4.7*
- 4 How does it differ from a mesh-based solver: what are the unknowns, and what takes the place of the mesh? *Liu ch. 1 – 2.1*
- 5 What is it used for: what is the operator form, which conditions make it well posed, and what is an inverse problem? *Liu ch. 2.2; Ex_07.1 notebook 02*

6 – 10 · PINN LOSS

- 6 What is the composite loss: which terms does it have, and where is each one evaluated? *Liu ch. 2.2.2*
- 7 How does soft enforcement impose a condition, and why is it met only approximately? *Liu ch. 2.2.2; Ex_07.1 notebook 01*
- 8 How is the loss tuned: what is a loss weight, physically, and what does gradient-norm balancing do? *Liu ch. 2.2.3; Ex_07.1 notebook 01*
- 9 How does hard enforcement differ from soft: what does the trial solution guarantee, and what does a hole cost? *Liu ch. 2.5.3, 2.9; Ex_07.1 notebook 02*
- 10 What is the loss used for across Part 2: what changes from block to block, and where are the collocation points placed? *Liu ch. 2.6; Ex_07.1 notebook 03*

NEXT

L7.2 — Fundamental PDEs

The same loop for every equation: only the residual and its conditions change.

BEFORE THE NEXT SESSION

read	Liu ch. 2.1 – 2.9
run	Ex_07.1 notebooks 00 to 02
bring	the wall error, soft and hard
think about	which conditions your problem carries

Part 2 has one method and many applications. This was the method.